```
    optimizer.zero_grad()
    output = model(x)
    loss = criterion(output, y) loss.backward()
    optimizer.step()

# Save the model
torch.save(model.state_dict(), 'simple_model.pth') # Load the model
loaded_model = SimpleModel()
loaded_model.load_state_dict(torch.load('simple_model.pth')) loaded_model.eval()
print("Loaded model prediction:", loaded_model(torch.tensor([[4.0]]))) PyTorch's state_dict provides a compact and efficient way to save only the essential parameters of the model.
```

Best Practices for Exporting Models

1. **Choose the Right Format**: Select a format based on compatibility and future use cases.
2. **Document the Model**: Include metadata such as model version, training data, and parameters.
3. **Test After Loading**: Always test the loaded model to ensure its integrity.
4. **Compress Large Models**: Use compression techniques to save storage and enhance portability.

Exporting and saving models is a fundamental practice in machine learning and AI workflows. By leveraging the capabilities of scikit-learn, TensorFlow, and PyTorch, developers can ensure their models are ready for deployment, collaboration, and further analysis. Embracing best practices in this area enhances the reproducibility, efficiency, and robustness of AI systems.

Building Flask APIs for AI Predictions

Flask is a lightweight and versatile Python web framework that makes it simple to create RESTful APIs. By integrating AI models with Flask, developers can efficiently serve predictions through HTTP endpoints. This capability is essential for building scalable, real-time prediction systems that can be accessed by various clients, such as web applications, mobile apps, or IoT devices.